

Solución - Trabajo Práctico N°1

Calidad del Software, Deuda Técnica y Code Smells

Parte A — Identificación de Problemas

Code Smell	Descripción
Long Method	El método procesar() realiza demasiadas tareas.
Duplicación	Existe lógica repetida para pacientes premium.
Condicionales complejos	Hay múltiples estructuras if anidadas.
Primitive Obsession	Uso excesivo de tipos primitivos.
Nombres poco descriptivos	Los nombres no expresan claramente intención.

Problemas de mantenibilidad

- Alta complejidad para agregar nuevas obras sociales.
- Difícil reutilización de lógica.
- Escasa separación de responsabilidades.

Ejemplos de deuda técnica

- Uso de Strings para representar obras sociales.
- Lógica centralizada en una única clase.

Calidad Interna vs Calidad Externa

Tipo	Ejemplo
Calidad Externa	El sistema agrega turnos correctamente para el usuario.
Calidad Interna	El código posee mala organización y baja mantenibilidad.

Conclusión: Un sistema puede funcionar correctamente para el usuario y aun así poseer mala calidad interna.

Refactor Aplicado

```
import java.util.ArrayList;
import java.util.List;

public class GestorTurnos {

    private final List<String> turnos =
        new ArrayList<>();

    public void registrarTurno(
        String nombrePaciente,
        int edad,
        String obraSocial,
        boolean urgente
    ) {
```

```

        if (!esPacienteValido(
            nombrePaciente,
            edad
        )) {
            return;
        }

        mostrarCategoriaPaciente(obraSocial);

        String turno = construirTurno(
            nombrePaciente,
            edad,
            obraSocial,
            urgente
        );

        turnos.add(turno);

        System.out.println("Turno agregado");
    }

    private boolean esPacienteValido(
        String nombrePaciente,
        int edad
    ) {
        return nombrePaciente != null
            && !nombrePaciente.isBlank()
            && edad > 0;
    }
}

```

Mejoras Aplicadas

Mejora	Beneficio
Extract Method	Reduce complejidad y mejora lectura.
Mejores nombres	Mayor claridad semántica.
Eliminación de duplicación	Facilita mantenimiento.
Separación de responsabilidades	Mayor cohesión.
Menor anidamiento	Mejor legibilidad.

Resultado Final: El sistema mantiene el mismo comportamiento funcional, pero ahora el código es más mantenible, legible, extensible y profesional.